

Intégrer un agent IA

dans un processus métier

Guide de mise en œuvre. Critères d'éligibilité, architecture, déploiement progressif et conformité.

Denis Lamard

Concepteur et intégrateur d'agents IA

github.com/denislamard/loom

Version 1.3, 2026

Sommaire

1. À qui s'adresse ce guide
2. La question préalable, agent ou workflow ?
3. La grille d'éligibilité
4. Cartographier le processus réel
5. Architecture, confiner le non-déterminisme
6. Contrat de sortie et juge sémantique
7. Le déploiement progressif
8. Mesurer et maîtriser le coût
9. Conformité et protection des données
10. Ce qui casse en production
11. Ce qu'il ne faut pas automatiser
12. Checklist avant mise en production
13. À propos

1. À qui s'adresse ce guide

Ce document s'adresse aux personnes qui doivent décider s'il faut mettre un agent IA dans un processus métier, et qui seront tenues responsables du résultat. Direction technique, responsable des opérations, dirigeant de PME ou d'ETI.

Il ne présente pas une technologie mais une méthode. Comment évaluer si un processus est un bon candidat, comment intégrer l'agent sans casser l'existant, comment monter en autonomie sans prendre de risque, et comment mesurer ce que ça produit réellement.

Le fil conducteur est une boîte mail partagée, *contact@*, *sav@*, *commandes@*, parce que toutes les organisations en ont une et que le cas se transpose facilement. Ce n'est pas une étude de cas client. C'est un support pour dérouler la méthode, qui s'applique à l'identique à la relance commerciale, au traitement de commandes, au tri de tickets ou à la qualification de demandes entrantes, ou tous autres processus métier

Ce que vous trouverez ici

- Une grille d'éligibilité scorée, utilisable seul sur vos propres processus
- Les principes d'architecture qui séparent un prototype d'un système exploitable
- Un protocole de déploiement en 4 marches, avec des seuils chiffrés
- Un plan de mesure et de maîtrise du coût
- Les points de conformité à traiter avant la mise en production
- Les défaillances les plus fréquentes en production, et comment les éviter

2. La question préalable, agent ou workflow ?

Avant tout choix technique, cette question doit être tranchée : peut-on énumérer les branches du processus ?

Si les branches sont énumérables, faites un workflow

Avant tout choix technique, cette question doit être tranchée : peut-on énumérer les branches du processus ?

Autrement dit, peut-on lister à l'avance tous les cas de figure et ce qu'il faut faire dans chacun ? Pour une facture fournisseur, oui. Conforme au bon de commande, au-dessus du seuil de validation, fournisseur non référencé. Trois cas, écrits une fois pour toutes. Pour un message qui arrive sur *contact@*, non, et la liste s'allongera chaque semaine.

Si les cas ne sont pas énumérables, il faut un agent

Un message entrant peut contenir plusieurs demandes de natures différentes, arriver dans une langue variable, renvoyer implicitement à un échange d'il y a trois mois, ou

porter une pièce jointe qui change complètement le sens de la phrase. Aucune arborescence de règles ne couvre cet espace. Chaque tentative ajoute une couche, et au bout d'un an plus personne n'ose y toucher.

Le signal fiable tient en une observation. Si vous n'arrivez pas à écrire la liste exhaustive des cas d'entrée, et que cette liste s'allonge chaque semaine, vous êtes dans le domaine de l'agent.

On se trompe dans les deux sens. Un agent là où trois règles auraient suffi, et vous payez un appel de modèle par exécution pour un résultat moins fiable qu'un if. Des règles là où les cas ne se laissent pas lister, et le fichier de conditions grossit jusqu'à ce que plus personne n'ose y toucher.

Le cas mixte, qui est le plus fréquent

En pratique, la bonne architecture est presque toujours hybride. Un workflow déterministe pilote le processus de bout en bout et n'appelle l'agent que là où il faut interpréter du langage libre ou arbitrer entre des options qu'on ne peut pas lister. C'est le principe directeur de tout ce qui suit.

3. La grille d'éligibilité

Savoir qu'un agent est le bon outil ne dit pas encore sur quel processus le brancher. Plusieurs candidats remplissent la condition, et ils sont loin de se valoir. Quatre critères permettent de trancher :

- **Le volume**, parce qu'une intégration a un coût fixe qu'il faut amortir
- **La valeur unitaire**, parce qu'une erreur ne doit jamais faire perdre une affaire.
- **La réversibilité**, parce qu'il faut pouvoir réparer sans que personne ne s'en aperçoive.
- **La mesurabilité**, parce qu'un résultat qu'on ne peut pas montrer ne sera pas reconduit.

3.1 Le volume

Une intégration se paie une fois, quel que soit le nombre de cas qu'elle traitera ensuite. Cartographier le processus, développer, poser les garde-fous, passer en production, accompagner les équipes. Comptez plusieurs semaines de travail avant le premier cas traité en autonomie.

Ce coût-là ne bouge plus. C'est le volume qui décide de ce qu'il vous rapporte.

Imaginons un service qui reçoit 300 demandes par mois, 10 de traitement chacune. 50 heures par mois, 600 heures par an. Même en n'automatisant que la moitié, vous récupérez l'équivalent de deux mois de travail à temps plein, tous les ans. L'intégration est remboursée dès la première année et continue de produire ensuite.

Le même service qui reçoit 4 demandes par mois y consacre quarante minutes. Vous pouvez automatiser la totalité, vous aurez gagné 8 heures par an pour plusieurs semaines de travail investies. Personne ne fait ce calcul et ne le refait deux fois.

Le volume ne décide pas seulement de ce que l'agent rapporte. Il décide aussi de sa survie.

Un outil qui touche peu de monde ne trouve personne pour le porter. Le processus change, mais personne ne pense à prévenir. Un cas mal traité agace, sans qu'on prenne la peine de le signaler puisque le contourner à la main va plus vite. Six mois plus tard, l'agent tourne toujours, sauf que plus personne ne s'y fie.

3.2 La valeur unitaire

Aucun agent ne travaille sans erreur. Ce n'est pas un défaut de jeunesse qu'on corrigera dans quelques mois de production, c'est la nature de l'outil. La question n'est donc pas de savoir s'il se trompera, mais ce que la première erreur vous coûtera.

Un taux de 97 % sonne très bien en réunion. Sur trois cents cas par mois, il signifie 9 erreurs. Multipliez 9 par ce que coûte une erreur sur ce processus précis, et vous avez votre réponse.

9 réponses de premier niveau mal aiguillées, ce sont 9 clients qui attendent un jour de plus. Ennuyeux, réparable dans la journée, sans conséquence durable. 9 devis chiffrés de trop bas ou pas assez détaillés, c'est un tout autre sujet. Une seule proposition partie 20 % sous le prix engage l'entreprise sur un chantier à perte, et les huit autres ne se voient même pas puisqu'elles ont été perdues sans qu'on sache pourquoi.

Le raisonnement tient donc en deux temps. Estimez le coût d'une erreur, en euros ou en confiance perdue. Multipliez par le nombre d'erreurs que votre volume produira. Si le total dépasse le gain attendu, le processus est disqualifié, quelle que soit la qualité du système qu'on mettra autour.

Notez que ce critère se corrige. Une valeur unitaire élevée n'interdit pas l'agent, elle interdit l'agent autonome. Vous pouvez très bien lui confier la préparation d'un devis, le rassemblement des références de prix et la mise en forme, à condition qu'un humain valide le chiffre avant l'envoi. Vous gardez l'essentiel du temps gagné et l'erreur ne sort jamais.

3.3 La réversibilité

Le critère précédent mesure ce que coûte une erreur. Celui-ci mesure ce qu'on peut encore en faire.

La question n'est pas de savoir si l'action s'annule, presque aucune ne s'annule. Elle est de savoir combien de temps s'écoule entre l'erreur et le moment où quelqu'un d'extérieur la voit. Tant que personne n'a vu, vous corrigez. Une fois que le client a lu, vous ne corrigez plus, vous vous excusez.

Trois situations très différentes se cachent derrière le mot.

Un brouillon en attente de validation ne sort jamais du système. L'erreur est visible en interne, corrigée en trente secondes, et personne d'autre n'en saura rien. C'est la situation confortable.

Un message d'accusé de réception parti au mauvais destinataire est irrattrapable, mais anodin. Vous ne pouvez rien annuler, sauf que le préjudice est nul et qu'un mot d'excuse referme le sujet.

Une mise en demeure adressée à un client à jour de ses paiements est irrattrapable et grave. Le destinataire l'a lue, il a compris qu'on le prenait pour un mauvais payeur, et aucun correctif ne rattrape ce qu'il pense désormais de vous. C'est le seul des trois cas qui disqualifie le processus.

Le plus utile à retenir, c'est que la réversibilité se conçoit, elle ne se constate pas. Vous ne la subissez pas, vous la fabriquez.

Un mode brouillon rend réversible à peu près n'importe quelle rédaction, au prix d'un clic humain. Un délai de quelques minutes entre la décision et l'envoi laisse le temps d'annuler, et coûte moins cher qu'une validation systématique. Une liste de motifs qui bloquent automatiquement, mention d'un litige, montant au-delà d'un seuil, client signalé, retire du périmètre les cas où l'erreur ferait mal. Une action réversible par construction, comme créer un brouillon au lieu d'envoyer, ou proposer un créneau au lieu de le réserver, obtient le même résultat sans validation du tout.

Autrement dit, un mauvais score sur ce critère n'est pas une fin de non-recevoir. C'est une contrainte de conception, et c'est en général la moins coûteuse à satisfaire.

3.4 La mesurabilité

Les trois critères précédents décident si l'agent va fonctionner. Les 3 critères précédents décident si l'agent va marcher. Celui-ci décide si vous pourrez prouver qu'il marche.

La différence compte plus qu'il n'y paraît. Un agent qui fonctionne mais dont personne ne sait chiffrer l'apport se fait couper au premier arbitrage budgétaire, en même temps que

les licences qu'on n'utilise plus. Personne ne le défendra, non par mauvaise foi, mais parce qu'aucun argument n'est disponible.

Un bon indicateur remplit 3 conditions :

- **Il est disponible avant le branchement.** Soit il existe déjà dans vos outils, soit vous le relevez pendant les semaines qui précèdent. Sans point de départ, aucune comparaison ne sera possible ensuite.
- **Il réagit vite.** Quelques semaines, pas 3 ans. Un indicateur qui met un an à bouger ne vous servira pas au moment où on vous demandera si ça marche.
- **Il est attribuable à l'agent.** Quand il progresse, vous devez pouvoir dire que c'est grâce à lui, sans que six autres facteurs aient varié en même temps.

Le délai de première réponse, autrement dit le temps qui sépare l'arrivée d'une demande de la première réponse envoyée au client, coche les 3. Il est déjà dans l'historique de votre outil, il bouge dès la première semaine, et rien d'autre que l'agent n'a pu le faire bouger.

Le taux de satisfaction client, non. Il varie aussi avec la qualité du produit, les délais de livraison et l'humeur de celui qui répond au questionnaire. Le mois où il progresse, vous ne saurez jamais dire ce qui revient à l'agent.

Le chiffre d'affaires, encore moins. Il bouge trop lentement, et une bonne année commerciale suffira à masquer complètement l'effet que vous cherchez à démontrer.

Attention au piège du chiffre trop beau. Le temps gagné est l'indicateur qu'on met partout, et c'est le plus faible. Personne n'a été licencié parce qu'un agent traite les demandes de premier niveau, donc le gain n'apparaît nulle part dans les comptes. Ce qui se voit vraiment, c'est ce que les équipes ont fait du temps récupéré, ou le volume absorbé sans embaucher. Formulez-le ainsi dès le départ, vous vous épargnerez une discussion pénible à la fin de l'année.

Un processus qui ne produit aucun chiffre exploitable n'est pas forcément à exclure. Il est à repousser, le temps d'installer la mesure. C'est souvent quelques jours de travail, et ça vaut mieux que de découvrir dans onze mois qu'on ne peut rien démontrer.

3.5 La grille scorée

Notez chaque critère de 0 à 3 pour le processus évalué.

Critère	0	1	2	3
Volume	Moins de 10 / mois	10 à 50 / mois	50 à 200 / mois	Plus de 200 / mois
Valeur unitaire	Affaire structurante	Contrat significatif	Transaction courante	Tâche de traitement
Réversibilité	Irréversible	Réparable à un coût élevé	Réparable sous 24 h	Sans effet externe direct
Mesurabilité	Aucun indicateur	Indicateur qualitatif	Mesuré ponctuellement	Suivi en continu

Voici comment je lis le score obtenu.

- **10 à 12.** Candidat prioritaire, c'est par là qu'il faut commencer.
- **7 à 9.** Candidat viable, à condition d'identifier le critère faible et de le compenser par conception, avec une validation humaine ou un périmètre restreint.
- **4 à 6.** À repousser. Le processus n'est pas mûr, ou l'agent n'est pas le bon outil.
- **0 à 3.** Ne pas automatiser. Un zéro sur la réversibilité est d'ailleurs éliminatoire à lui seul, quel que soit le total.

Quand il faut décider vite, j'utilise ce raccourci. Si l'agent se trompe le mardi, puis-je réparer le mercredi sans perte d'argent ni de confiance ?

3.6 Un exemple d'application

3 processus fréquemment cités, passés à la grille.

Processus	Vol.	Valeur	Réversib.	Mesure	Total	Décision
Tri et réponse de premier niveau sur boîte partagée	3	3	3	3	12	Prioritaire
Relance de devis sans réponse	2	2	3	3	10	Prioritaire
Chiffrage d'une proposition commerciale	1	0	0	2	3	À exclure

Le troisième cas illustre bien la limite. Techniquement faisable, commercialement séduisant, mais l'erreur est irréversible et le chiffage constitue le savoir-faire propre de l'entreprise. Ce n'est pas un candidat.

4. Cartographier le processus réel

Le processus documenté et le processus vécu divergent toujours. La procédure indique que chaque demande est qualifiée puis affectée. Dans les faits, une personne connaît les 20 clients qui écrivent le plus et court-circuite tout le circuit. C'est cette version-là qu'il faut modéliser.

Les quatre éléments à extraire

- **Le déclencheur.** Quel événement exact lance le processus, et par quel canal il arrive dans le système.
- **Les points de décision.** Qui décide quoi, sur la base de quelles informations, avec quelle marge d'erreur tolérée, et ce qui se passe aujourd'hui en cas de doute.
- **Les sources de vérité.** Où vit la donnée, qui a le droit de l'écrire, et laquelle fait foi quand deux systèmes divergent.
- **La définition de « terminé ».** Ce qui doit exister pour que le cas soit clos, un statut, un enregistrement, une trace.

La méthode qui fonctionne, c'est d'observer plutôt que d'interroger. Une demi-journée passée à regarder quelqu'un traiter des cas réels apprend plus que trois réunions de cadrage. Et sur chaque cas observé, posez la question qui rapporte le plus. Celui-là, pourquoi l'avez-vous traité autrement ? Les exceptions sont l'information utile.

Sans ce travail, l'agent fonctionne en démonstration et casse en production sur la branche que personne n'avait mentionnée, parce qu'elle allait de soi pour ceux qui la pratiquent tous les jours.

5. Architecture, confiner le non-déterminisme

C'est le cœur du sujet, et ce qui sépare un prototype d'un système exploitable. 3 principes, non négociables.

5.1 L'orchestrateur pilote, l'agent décide à un endroit précis

Le modèle n'est pas le chef d'orchestre. C'est un composant appelé à un moment donné, dont la sortie est vérifiée avant d'avoir le moindre effet. Le déroulé du processus, lui, reste du code déterministe qu'on peut lire, tester et rejouer à l'identique.

La conséquence pratique est une séparation stricte entre les outils de lecture et les outils d'écriture. Les lectures sont libres et parallélisables, qu'il s'agisse de consulter une fiche client, de chercher dans une base documentaire ou d'extraire des entités. Les écritures, elles, passent toutes par une couche unique qui valide, journalise et applique une clé d'idempotence.

5.2 Les règles métier vivent dans le code, jamais dans le prompt

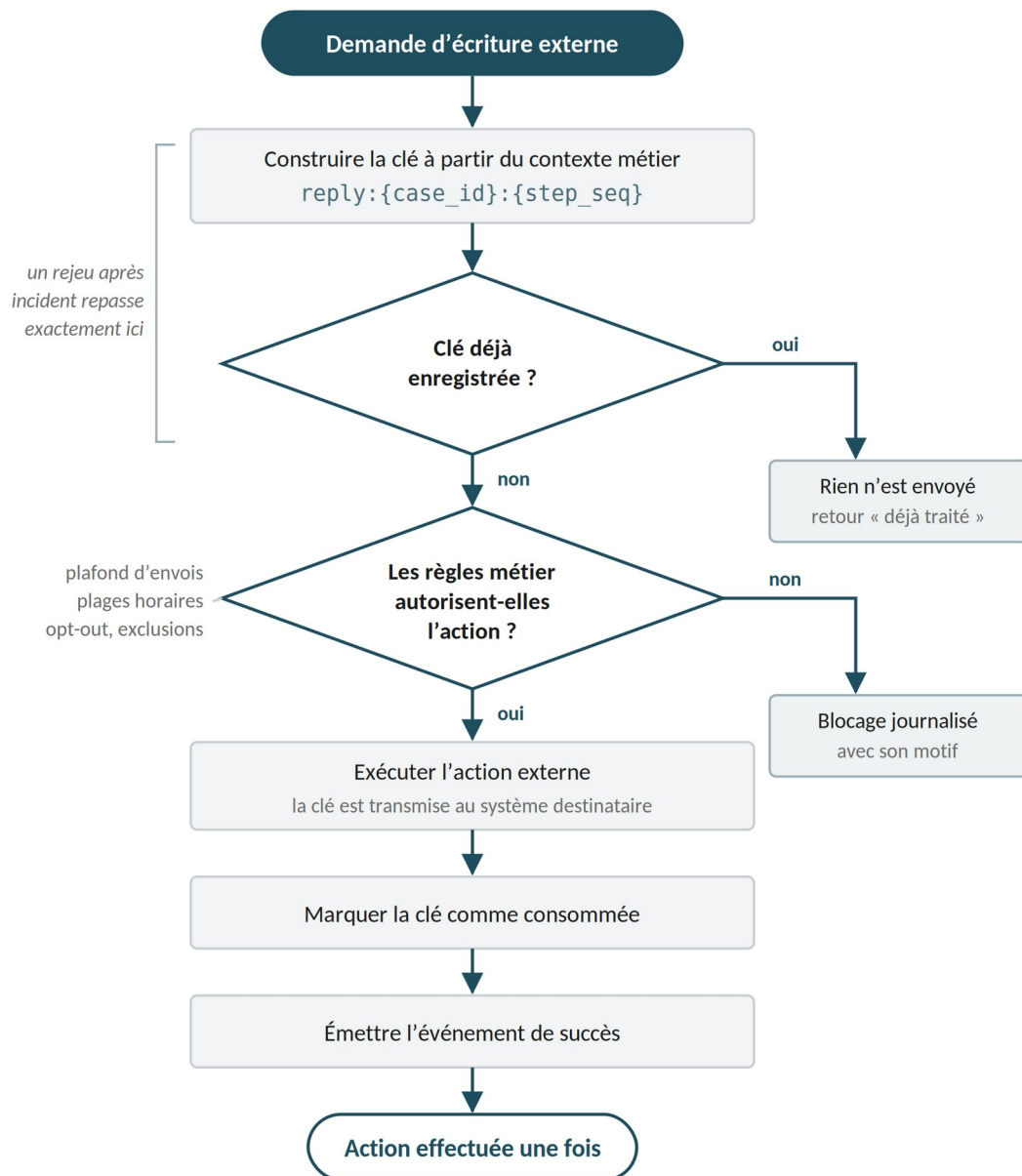
Plafond d'envois, plages horaires, listes d'exclusion, opt-out, interdiction d'engager un prix ou un délai, obligations réglementaires. Un modèle de langage ne peut pas être le seul gardien d'une contrainte légale ou contractuelle. Il n'offre aucune garantie, seulement une probabilité élevée de bien se comporter.

Une règle écrite dans un prompt reste une consigne. Une règle écrite dans le code devient une contrainte. La différence apparaît précisément le jour où un cas inhabituel se présente, et c'est ce jour-là qu'elle compte.

5.3 Toute écriture externe est idempotente

Un système de production redémarre, rejoue une file de messages, reprend après incident. Sans clé d'idempotence déterministe, le destinataire reçoit 3 fois la même relance et vous perdez en une matinée la confiance construite en six mois.

Le point de vigilance porte sur la clé elle-même. Elle doit être dérivée du contexte métier et jamais générée aléatoirement, sinon le rejeu produit une nouvelle clé et l'action se duplique quand même.



Chemin d'écriture idempotent. Toute action externe traverse ces deux décisions avant de produire le moindre effet.

Trois choses méritent d'être notées dans ce schéma. La vérification d'idempotence précède tout le reste, y compris l'évaluation des règles. La politique métier est tranchée par du code et non par le modèle. Et chaque issue émet un événement exploitable, le blocage compris, qui se révèle souvent l'information la plus intéressante quand on cherche à comprendre le comportement du système.

5.4 Le branchement au système d'information existant

Un agent n'a d'intérêt que s'il accède aux données réelles, CRM, ERP, outil de ticketing, base documentaire. On l'y branche en lecture par des interfaces explicites, sans réécrire le système d'information, et on limite les écritures au strict nécessaire.

Ces contraintes reviennent à l'identique sur chaque intégration. Plafonnement du coût, validation des sorties, escalade tracée, mémoire de session, observabilité. Je les ai factorisées dans un harness d'orchestration que je maintiens en open source, disponible sur github.com/denislamard/loom, plutôt que de les réécrire à chaque projet. La méthode décrite dans ce guide ne dépend pas de cet outil. L'outil fait simplement gagner les semaines qu'on passe habituellement à recoder la même plomberie.

6. Contrat de sortie et juge sémantique

Entre la sortie du modèle et l'action, il faut une vérification. C'est la couche la plus souvent absente des intégrations, et pourtant celle qui détermine si le système pourra un jour fonctionner sans supervision permanente.

6.1 Premier niveau, le contrat structurel

Une validation mécanique, déterministe et peu coûteuse. Schéma attendu, champs obligatoires, longueur minimale et maximale, motifs interdits, format des références. Elle attrape les défauts grossiers et les hallucinations de format.

```
contract:
  schema:    ReplyDraft    # objet typé, champs obligatoires
  max_chars: 1200
  must_contain: ["{case_ref}"] # la référence du dossier
  must_not_match:
    - "\\b\\d+\\s?(€|EUR)"    # aucun montant
    - "(?i)sous \\d+ (jours|heures)" # aucun engagement de délai
  language:  same_as_input
```

Ce que le contrat structurel ne saura pas faire, en revanche, c'est juger si la réponse est pertinente, si le ton est adapté, ou si elle contient un engagement implicite formulé autrement que par les motifs prévus.

6.2 Second niveau, le juge sémantique

Un second modèle, distinct du modèle générateur, évalue la sortie sur ce qu'aucune expression régulière ne saura capturer. Il ne réécrit rien, il rend un verdict motivé.

Voici les critères que je fais évaluer systématiquement.

- La réponse traite-t-elle effectivement la demande posée, et toutes les demandes si le message en contenait plusieurs ?
- Contient-elle une affirmation qui n'est pas étayée par les données consultées ?
- Prend-elle un engagement de délai, de prix ou de responsabilité, sous quelque formulation que ce soit ?

- Le registre est-il conforme à la charte de communication ?

En cas d'écart, une passe de réparation est déclenchée avec le motif du rejet en entrée. Si la seconde tentative échoue également, le cas part vers un humain et n'est jamais envoyé en l'état. Le nombre de tentatives reste plafonné, sans quoi une boucle de réparation peut consommer un budget entier sur un seul cas pathologique.

6.3 Le coût du contrôle

Un juge sémantique ajoute un appel par exécution. Sur un cas de traitement de message, il représente typiquement 15 à 25 % du coût total, un ordre de grandeur qu'il faut mesurer sur votre propre volumétrie. Deux leviers le maintiennent bas. Confier le rôle de juge à un modèle rapide et peu coûteux, et ne le déclencher que sur les sorties destinées à une action externe plutôt que sur chaque étape interne.

Un système sans couche de vérification ne dépassera jamais le stade du brouillon validé par un humain. C'est la contrainte qui plafonne la valeur de la plupart des intégrations que je vois passer.

7. Le déploiement progressif

Personne ne branche un agent en autonomie totale le premier jour. On monte quatre marches, et on ne monte que sur des chiffres.

Marche	Ce que fait l'agent	Condition de passage
1. Shadow	Traite les cas réels, écrit ses décisions, n'envoie rien	Concordance avec la décision humaine supérieure à 85 % sur 100 cas
2. Brouillon validé	Prépare, l'humain valide en un clic	Taux de correction humaine sous 5 % sur 100 cas consécutifs
3. Autonome sous garde-fous	Agit seul sur un segment restreint, escalade au moindre doute	Taux d'escalade stable, aucun incident sur 30 jours
4. Autonome	Agit seul sur le périmètre éprouvé	Aucune, c'est la dernière marche

7.1 Le mode shadow, en détail

C'est la phase la plus instructive, et celle qu'on saute le plus souvent. L'agent est branché sur le flux réel, il produit ses décisions et les enregistre, mais aucune action ne part. On compare ensuite, cas par cas, à ce que l'humain a fait.

- **Combien de temps.** Deux à quatre semaines, ou le temps de couvrir 100 à 200 cas réels. Le critère de volume prime sur le critère de durée.
- **Ce qu'on compare.** La classification d'abord, puis les données extraites, la décision d'action et enfin le contenu produit. Cet ordre compte, parce qu'un écart de classification explique souvent tous les autres.
- **Ce qu'on en tire.** Les cas de désaccord deviennent la base des tests de régression. Ce sont eux qui font progresser le système, pas les cas réussis.

Un désaccord n'est d'ailleurs pas toujours une erreur de l'agent. Une part significative des écarts révèle une inconstance humaine, deux personnes traitant le même cas différemment, ou une règle que personne n'avait jamais écrite. Ces découvertes ont une valeur propre, indépendamment du projet d'automatisation, et elles justifient souvent à elles seules le temps passé en shadow.

7.2 L'escalade doit aboutir quelque part

Un agent qui escalade vers un tableau de bord que personne n'ouvre n'escalade pas. Il abandonne silencieusement. L'escalade doit donc emprunter le canal déjà utilisé par les équipes, la messagerie interne ou l'outil de ticketing existant, et surtout pas une interface supplémentaire.

Chaque nouvel outil imposé réduit l'adoption. La règle est simple, l'agent s'insère dans les outils existants, il n'en ajoute pas.

Les conditions d'escalade se définissent explicitement et ne se laissent pas à l'appréciation du modèle. Confiance sous un seuil, cas hors du périmètre déclaré, montant au-delà d'une limite, mention d'un litige ou d'un terme juridique, deuxième échec de réparation.

8. Mesurer et maîtriser le coût

8.1 La baseline, avant le branchement

À collecter impérativement avant toute mise en service. Sans ces chiffres pris en amont, vous n'aurez rien à présenter au moment de reconduire le budget, et aucun moyen de repérer une régression.

Indicateur	Comment le relever
Volume traité par période	Export de l'outil source sur 3 mois glissants
Délai de première réponse	Médiane et 90e centile, la moyenne ne dit rien
Temps humain consacré	Estimation déclarative croisée avec le volume

Indicateur	Comment le relever
Taux de cas non traités	Cas restés sans action au-delà du seuil interne
Indicateur métier terminal	Conversion, recouvrement ou satisfaction selon le processus

8.2 L'instrumentation technique

Chaque exécution émet des événements structurés, exploitables sans retraitement. Voici le minimum utile.

```
{
  "ts": "2026-06-12T09:14:22Z",
  "run": "r-8842",
  "step": "classify",
  "role": "triage",
  "model": "haiku",
  "tokens_in": 1840,
  "tokens_out": 96,
  "cost_eur": 0.0021,
  "latency_ms": 740,
  "status": "ok"
}

{
  "ts": "2026-06-12T09:14:26Z",
  "run": "r-8842",
  "step": "judge",
  "verdict": "repair",
  "reason": "engagement_delai",
  "attempt": 1
}

{
  "ts": "2026-06-12T09:14:31Z",
  "run": "r-8842",
  "step": "send_reply",
  "status": "escalated",
  "reason": "confidence_below_threshold"
}
```

Cinq indicateurs se suivent en continu. Le coût par unité traitée, la latence de bout en bout, le taux d'escalade, le taux de correction humaine et le taux de déclenchement de la réparation. Une dérive sur l'un d'eux précède toujours un incident visible, ce qui vous laisse le temps d'intervenir avant que quelqu'un ne s'en plaigne.

8.3 Le routage par rôle

C'est le principal levier de coût, et il est structurel plutôt que conjoncturel. Un même processus mobilise des tâches d'exigence très inégale. Classer, extraire, résumer, décider, rédiger. Leur confier le même modèle revient à payer le prix de la tâche la plus difficile sur l'ensemble du volume.

En pratique, un modèle rapide et peu coûteux assure la classification, l'extraction d'entités et le rôle de juge, pendant qu'un modèle plus puissant n'intervient que sur la rédaction destinée au client et sur les arbitrages complexes. Selon la distribution des cas, l'écart entre une architecture mono-modèle et une architecture routée se situe couramment dans un rapport de trois à cinq.

8.4 Le plafond

Un budget maximal par période, au-delà duquel le système s'arrête et alerte plutôt que de dériver. C'est la première question posée en comité d'investissement, et la réponse doit être un mécanisme technique plutôt qu'une intention.

Deux protections complémentaires méritent d'être en place. Un plafond de tokens par exécution, qui empêche un cas pathologique de consommer un budget entier. Et une compaction de l'historique de session au-delà d'un seuil, sans laquelle le coût par échange croît continûment jusqu'à devenir absurde.

9. Conformité et protection des données

Ces points bloquent régulièrement les projets au moment de la mise en production, alors qu'ils se traitent sans difficulté quand on les anticipe. Ce qui suit ne constitue pas un avis juridique. Ce sont les questions à poser à votre conseil ou à votre DPO.

9.1 Sous-traitance et localisation

Un fournisseur de modèle qui traite des données personnelles pour votre compte est un sous-traitant au sens du RGPD. Il faut donc un contrat de sous-traitance, une inscription au registre des traitements, et une vérification du lieu d'hébergement ainsi que du mécanisme de transfert hors Union européenne le cas échéant. Vérifiez aussi la politique de rétention du fournisseur et l'existence d'une option de non-utilisation des données à des fins d'entraînement.

9.2 Minimisation

N'envoyez au modèle que ce qui est nécessaire à la tâche. C'est une obligation réglementaire, et c'est accessoirement une bonne pratique technique, puisque moins de contexte signifie moins de coût, moins de latence et moins de dispersion. Quand la tâche ne requiert pas l'identité, le masquage des identifiants directs avant l'appel avec réinjection après coup reste une mesure simple et très efficace.

9.3 Décision automatisée

Le RGPD encadre les décisions entièrement automatisées qui produisent des effets juridiques ou significatifs sur une personne. La plupart des cas d'usage décrits ici n'entrent pas dans ce champ, qu'il s'agisse de trier, de rédiger un brouillon ou de relancer. Un refus automatisé, une résiliation ou une décision d'octroi, si. La règle de prudence tient en une phrase. Dès qu'une décision affecte les droits d'une personne, elle passe par un humain.

9.4 Traçabilité

Conservez, pour chaque décision produite, les entrées consultées, la sortie générée, le verdict de validation et l'identité de l'humain qui a validé le cas échéant. Cette journalisation sert trois usages distincts, répondre à une réclamation, démontrer la conformité et diagnostiquer une régression. Définissez sa durée de conservation en même temps que vous la mettez en place, sinon elle ne sera jamais définie.

9.5 Transparence et cadre européen

Informez les personnes concernées lorsqu'elles interagissent avec un système automatisé, et prévoyez un moyen simple d'accéder à un interlocuteur humain. Au-delà du RGPD, le règlement européen sur l'intelligence artificielle introduit des obligations dont l'application s'échelonne dans le temps et dépend de la catégorie de risque du système. L'état exact des échéances applicables à votre cas se vérifie auprès de votre conseil au moment du projet.

10. Ce qui casse en production

Voici les défaillances que je rencontre le plus souvent, avec la parade correspondante.

Défaillance	Cause	Parade
Action dupliquée après incident	Clé d'idempotence aléatoire ou absente	Clé dérivée du contexte métier, vérifiée avant l'action
Cas non prévu traité comme un cas normal	Branche non documentée à la cartographie	Périmètre déclaré explicite, escalade par défaut hors périmètre
Coût qui dérive sans alerte	Historique de session non compacté	Compaction au-delà d'un seuil, plafond par exécution
Dérive silencieuse de la qualité	Aucun test de régression sur les cas réels	Jeu de cas issus du mode shadow, rejoué à chaque évolution
Escalade sans destinataire	Notification vers un outil non consulté	Canal existant des équipes, accusé de prise en charge
Régression après changement de modèle	Absence de jeu d'évaluation figé	Évaluation automatisée exécutée avant toute bascule
Réponse hors sujet sur message multi-demandes	Traitement de la première demande seulement	Extraction explicite des intentions, contrat de couverture

Un constat traverse ce tableau. Presque aucune de ces défaillances ne relève de la qualité du modèle. Elles relèvent toutes de l'ingénierie qu'on met autour. C'est là que se joue la mise en production, et c'est donc là que se concentre l'effort.

11. Ce qu'il ne faut pas automatiser

Une intégration réussie se reconnaît autant à ce qu'elle laisse de côté qu'à ce qu'elle prend en charge.

- **Les litiges et réclamations engageant une responsabilité.** Volume faible, chaque cas est singulier, et une formulation maladroite produit un dommage qui dure.
- **Les décisions tarifaires et le chiffrage.** Valeur unitaire élevée, erreur irréversible, et surtout, c'est le savoir-faire propre de l'entreprise.
- **Les résiliations et ruptures contractuelles.** Effet juridique direct, aucune réversibilité possible.
- **Le diagnostic technique engageant une garantie.** La responsabilité ne se délègue pas à un système probabiliste.
- **La négociation.** L'arbitrage doit rester humain, même quand l'agent en prépare parfaitement les éléments.

Ce ne sont pas des limites techniques. Un modèle actuel produit sur ces sujets des sorties souvent convaincantes, et c'est précisément le problème. Ce sont des limites de conception. Automatiser ce qui fait la valeur d'une équipe revient à détruire la valeur qu'on prétend augmenter.

Un agent bien posé retire la charge répétitive et rend du temps de décision. Il ne remplace pas la décision.

12. Checklist avant mise en production

À passer intégralement avant d'ouvrir le système sur le flux réel.

1. Le processus obtient au moins 7 à la grille d'éligibilité, sans zéro sur la réversibilité.
2. Le choix entre agent et workflow est motivé par écrit.
3. Le processus réel est cartographié, déclencheur, décisions, sources de vérité, état final attendu.
4. Les règles métier sont implémentées dans le code, et aucune ne repose uniquement sur le prompt.
5. Toute écriture externe dispose d'une clé d'idempotence déterministe.
6. Un contrat de sortie structurel est défini et testé, motifs interdits compris.
7. Un juge sémantique est en place, avec un nombre de réparations plafonné.
8. Les conditions d'escalade sont explicites, et l'escalade aboutit à un canal réellement consulté.

9. La baseline métier est relevée et datée.
10. L'instrumentation émet coût, latence, escalade, correction et réparation.
11. Un plafond de budget et un plafond par exécution sont actifs.
12. Le mode shadow a couvert au moins 100 cas réels et les désaccords ont été analysés.
13. Un jeu de tests de régression est constitué à partir de ces désaccords.
14. Le contrat de sous-traitance, le registre et la politique de rétention sont en place.
15. Le périmètre non automatisé est écrit et connu des équipes.
16. Une procédure d'arrêt d'urgence existe et a été testée.

13. À propos

Je conçois et intègre des agents IA en production. Les principes décrits dans ce guide sont mis en œuvre dans Loom, un harness d'orchestration d'agents en Python que je maintiens en open source. Orchestration multi-modèles, outils exécutés en parallèle, validation des sorties, comptabilité et plafonnement du coût, mémoire de session et observabilité.

Code disponible : github.com/denislamard/loom

Ce document peut être partagé librement. Il ne constitue pas un avis juridique.